Open in Colab

T81-558: Applications of Deep Neural Networks

Module 13: Advanced/Other Topics

- Instructor: Jeff Heaton, McKelvey School of Engineering, Washington University in St. Louis
- For more information visit the class website.

Module 13 Video Material

- Part 13.1: Flask and Deep Learning Web Services [Video] [Notebook]
- Part 13.2: Interrupting and Continuing Training [Video] [Notebook]
- Part 13.3: Using a Keras Deep Neural Network with a Web Application [Video] [Notebook]
- **Part 13.4: When to Retrain Your Neural Network** [Video] [Notebook]
- Part 13.5: Tensor Processing Units (TPUs) [Video] [Notebook]

Part 13.4: When to Retrain Your Neural Network

Dataset drift is a problem frequently seen in real-world applications of machine learning. Academic problems that courses typically present in school assignments usually do not experience this problem. For a class assignment, your instructor provides a single data set representing all of the data you will ever see for a task. In the real world, you obtain initial data to train your model; then, you will acquire new data over time that you use your model to predict.

Consider this example. You create a startup company that develops a mobile application that helps people find jobs. To train your machine learning model, you collect attributes about people and their careers. Once you have your data, you can prepare your neural network to suggest the best jobs for individuals.

Once your application is released, you will hopefully obtain new data. This data will come from job seekers using your app. These people are your customers. You have x values (their attributes), but you do not have y-values (their jobs). Your customers have come to you to find out what their be jobs will be. You will provide the customer's

attributes to the neural network, and then it will predict their jobs. Usually, companies develop neural networks on initial data and then use the neural network to perform predictions on new data obtained over time from their customers.

Your job prediction model will become less relevant as the industry introduces new job types and the demographics of your customers change. However, companies must look if their model is still relevant as time passes. This change in your underlying data is called dataset drift. In this section, we will see ways that you can measure dataset drift.

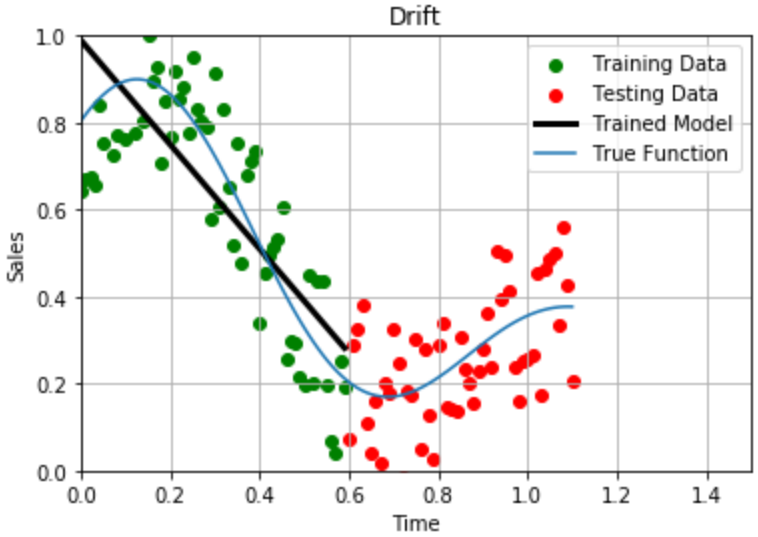
You can present your model with new data and see how its accuracy changes over time. However, to calculate efficiency, you must know the expected outputs from the model (y-values). You may not know the correct outcomes for new data that you are obtaining in real-time. Therefore, we will look at algorithms that examine the x-inputs and determine how much they have changed in distribution from the original x-inputs that we trained on. These changes are called dataset drift.

Let's begin by creating generated data that illustrates drift. We present the following code to create a chart that shows such drift.

In [4]:
```
import numpy as np

import matplotlib.pyplot as plot
from sklearn.linear_model import LinearRegression

def true_function(x):
    x2 = (x*8) - 1
    return ((np.sin(x2)/x2)*0.6)+0.3

#
x_train  = np.arange(0, 0.6, 0.01)
x_test  = np.arange(0.6, 1.1, 0.01)
x_true = np.concatenate( (x_train, x_test) )

#
y_true_train = true_function(x_train)
y_true_test = true_function(x_test)
y_true   = np.concatenate( (y_true_train, y_true_test) )

#
y_train = y_true_train + (np.random.rand(*x_train.shape)-0.5)*0.4
y_test  = y_true_test + (np.random.rand(*x_test.shape)-0.5)*0.4

#
lr_x_train = x_train.reshape((x_train.shape[0],1))
reg = LinearRegression().fit(lr_x_train, y_train)
reg_pred = reg.predict(lr_x_train)
print(reg.coef_[0])
print(reg.intercept_)

#
```

```
plot.xlim([0,1.5])
plot.ylim([0,1])
l1 = plot.scatter(x_train, y_train, c="g", label="Training Data")
l2 = plot.scatter(x_test, y_test, c="r", label="Testing Data")
l3, = plot.plot(lr_x_train, reg_pred, color='black', linewidth=3,
                label="Trained Model")
l4, = plot.plot(x_true, y_true, label = "True Function")
plot.legend(handles=[l1, l2, l3, l4])

#
plot.title('Drift')
plot.xlabel('Time')
plot.ylabel('Sales')
plot.grid(True, which='both')
plot.show()
```

```
-1.1979470956001936
0.9888340153211445
```



The "True function" represents what the data does over time. Unfortunately, you only have the training portion of the data. Your model will do quite well on the data that you trained it with; however, it will be very inaccurate on the new test data presented. The prediction line for the model fits the training data well but does not fit the est data well.

Preprocessing the Sberbank Russian Housing Market Data

The examples provided in this section use a Kaggle dataset named The Sberbank Russian Housing Market, which you can access from the following link.

- Sberbank Russian Housing Market

Because Kaggle provides datasets as training and test, we must load both of these files.

In [1]:
```
import os
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder

PATH = "/Users/jheaton/Downloads/sberbank-russian-housing-market"


train_df = pd.read_csv(os.path.join(PATH,"train.csv"))
test_df = pd.read_csv(os.path.join(PATH,"test.csv"))
```

I provide a simple preprocess function that converts all numerics to z-scores and all categoricals to dummies.

In [2]:
```
def preprocess(df):
    for i in df.columns:
        if df[i].dtype == 'object':
            df[i] = df[i].fillna(df[i].mode().iloc[0])
        elif (df[i].dtype == 'int' or df[i].dtype == 'float'):
            df[i] = df[i].fillna(np.nanmedian(df[i]))

    enc = LabelEncoder()
    for i in df.columns:
        if (df[i].dtype == 'object'):
            df[i] = enc.fit_transform(df[i].astype('str'))
            df[i] = df[i].astype('object')
```

Next, we run the training and test datasets through the preprocessing function.

In [3]:
```
preprocess(train_df)
preprocess(test_df)
```

Finally, we remove thr target variable. We are only looking for drift on the x (input data).

In [4]:
```
train_df.drop('price_doc',axis=1,inplace=True)
```

KS-Statistic

We will use the KS-Statistic to determine the difference in distribution between columns in the training and test sets. As a baseline, consider if we compare the same field to itself. In this case, we are comparing the **kitch_sq** in the training set. Because there is no difference in distribution between a field in itself, the p-value is 1.0, and the KS-Statistic statistic is 0. The P-Value is the probability of no difference between the two distributions. Typically some lower threshold is used for how low a P-Value is needed to reject the null hypothesis and assume there is a difference. The value of 0.05 is a standard threshold for p-values. Because the p-value is NOT below 0.05, we expect the two distributions to be the same. If the p-value were below the threshold, the **statistic**

value becomes interesting. This value tells you how different the two distributions are. A value of 0.0, in this case, means no differences.

In [5]:
```
from scipy import stats

stats.ks_2samp(train_df['kitch_sq'], train_df['kitch_sq'])
```

Out[5]:
```
Ks_2sampResult(statistic=-0.0, pvalue=1.0)
```

Now let's do something more interesting. We will compare the same field **kitch_sq** between the test and training sets. In this case, the p-value is below 0.05, so the **statistic** value now contains the amount of difference detected.

In [6]:
```
stats.ks_2samp(train_df['kitch_sq'], test_df['kitch_sq'])
```

Out[6]:
```
Ks_2sampResult(statistic=0.25829078867676714, pvalue=0.0)
```

Next, we pull the KS-Stat for every field. We also establish a boundary for the maximum p-value and how much of a difference is needed before we display the column.

In [7]:
```
for col in train_df.columns:
    ks = stats.ks_2samp(train_df[col], test_df[col])
    if ks.pvalue < 0.05 and ks.statistic>0.1:
        print(f'{col}: {ks}')
```

```
id: Ks_2sampResult(statistic=1.0, pvalue=0.0)
timestamp: Ks_2sampResult(statistic=0.8982081426022823, pvalue=0.0)
life_sq: Ks_2sampResult(statistic=0.2255084471628891, pvalue=7.29401465948424e-271)
max_floor: Ks_2sampResult(statistic=0.17313454154786817, pvalue=7.82000315371674e-16
0)
build_year: Ks_2sampResult(statistic=0.3176883950430345, pvalue=0.0)
num_room: Ks_2sampResult(statistic=0.1226755470309048, pvalue=1.8622542043144584e-8
0)
kitch_sq: Ks_2sampResult(statistic=0.25829078867676714, pvalue=0.0)
state: Ks_2sampResult(statistic=0.13641341252952505, pvalue=2.1968159319271184e-99)
preschool_quota: Ks_2sampResult(statistic=0.2364160801236304, pvalue=1.1710777340471
466e-297)
school_quota: Ks_2sampResult(statistic=0.25657342859882415, pvalue=0.0)
raion_build_count_with_material_info: Ks_2sampResult(statistic=0.19083554469945835,
pvalue=4.2830715478540455e-194)
build_count_block: Ks_2sampResult(statistic=0.2085099875571384, pvalue=1.24947259638
39073e-231)
build_count_wood: Ks_2sampResult(statistic=0.20880892257287548, pvalue=2.71212095640
49356e-232)
build_count_brick: Ks_2sampResult(statistic=0.20707302040295733, pvalue=1.8727526843
617417e-228)
build_count_monolith: Ks_2sampResult(statistic=0.1779053203005685, pvalue=9.70733704
9927237e-169)
raion_build_count_with_builddate_info: Ks_2sampResult(statistic=0.19083554469945835,
pvalue=4.2830715478540455e-194)
build_count_1946-1970: Ks_2sampResult(statistic=0.2120848849003817, pvalue=1.2611967
894038327e-239)
build_count_1971-1995: Ks_2sampResult(statistic=0.19318717795964874, pvalue=6.749018
351478723e-199)
build_count_after_1995: Ks_2sampResult(statistic=0.1899788049625577, pvalue=2.326885
447792135e-192)
cafe_sum_500_min_price_avg: Ks_2sampResult(statistic=0.43416513954613944, pvalue=0.
0)
cafe_avg_price_500: Ks_2sampResult(statistic=0.4337483600913839, pvalue=0.0)
cafe_sum_1000_min_price_avg: Ks_2sampResult(statistic=0.20459725920896277, pvalue=4.
926006714051654e-223)
cafe_sum_1000_max_price_avg: Ks_2sampResult(statistic=0.2025042643599122, pvalue=1.6
73760717424549e-218)
cafe_avg_price_1000: Ks_2sampResult(statistic=0.20399591976318965, pvalue=9.97989811
3798837e-222)
cafe_sum_1500_min_price_avg: Ks_2sampResult(statistic=0.13264580849650315, pvalue=5.
402810864486807e-94)
cafe_avg_price_1500: Ks_2sampResult(statistic=0.13316992134991978, pvalue=9.81160594
279586e-95)
cafe_sum_2000_min_price_avg: Ks_2sampResult(statistic=0.10808898569668424, pvalue=1.
4808059430311726e-62)
cafe_sum_2000_max_price_avg: Ks_2sampResult(statistic=0.10732529051140638, pvalue=1.
1100804327460878e-61)
cafe_avg_price_2000: Ks_2sampResult(statistic=0.1081218037860151, pvalue=1.357575991
1857293e-62)
```

Detecting Drift between Training and Testing Datasets by Training

Sample the training and test into smaller sets to train. We want 10K elements from each; however, the test set only has 7,662, so we only sample that amount from each side.

In [8]:
```
SAMPLE_SIZE = min(len(train_df),len(test_df))
SAMPLE_SIZE = min(SAMPLE_SIZE,10000)
print(SAMPLE_SIZE)
```

```
7662
```

We take the random samples from the training and test sets and add a flag called **source_training** to tell the two apart.

In [9]:
```
training_sample = train_df.sample(SAMPLE_SIZE, random_state=49)
testing_sample = test_df.sample(SAMPLE_SIZE, random_state=48)

# Is the data from the training set?
training_sample['source_training'] = 1
testing_sample['source_training'] = 0
```

Next, we combine the data that we sampled from the training and test data sets and shuffle them.

In [13]:
```
# Build combined training set
combined = testing_sample.append(training_sample)
combined.reset_index(inplace=True, drop=True)

# Now randomize
combined = combined.reindex(np.random.permutation(combined.index))
combined.reset_index(inplace=True, drop=True)
```

We will now generate x and y to train. We attempt to predict the **source_training** value as y, which indicates if the data came from the training or test set. If the model successfully uses the data to predict if it came from training or testing, then there is likely drift. Ideally, the train and test data should be indistinguishable.

In [12]:
```
# Get ready to train
y = combined['source_training'].values
combined.drop('source_training',axis=1,inplace=True)
x = combined.values

y
```

Out[12]:
```
array([1, 1, 1, ..., 1, 0, 0])
```

We will consider anything above a 0.75 AUC as having a good chance of drift.

In [15]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

model = RandomForestClassifier(n_estimators = 60, max_depth = 7,
```

```
    min_samples_leaf = 5)
lst = []

for i in combined.columns:
    score = cross_val_score(model,pd.DataFrame(combined[i]),y,cv=2,
                            scoring='roc_auc')
    if (np.mean(score) > 0.75):
        lst.append(i)
        print(i,np.mean(score))
```

```
id 1.0
timestamp 0.9601862111975688
full_sq 0.7966785611424911
life_sq 0.8724218330166038
build_year 0.8004825176688191
kitch_sq 0.9070093804672634
cafe_sum_500_min_price_avg 0.8435920036035689
cafe_avg_price_500 0.8453533835344671
```

In [ ]:

```

```